Beyond Chatbots: AI Agents

Peter Ganong and Maggie Shi

February 23, 2026

By the end of this lecture, you will know…

- How to start and configure **an agent** for a data analysis project
- Two concrete examples of agents doing real research tasks
- What **skills** are and how to install them safely
- The key **dangers** of AI agents and how to protect yourself

Motivation

Roadmap

- What this lecture is not about
- How do agents differ from chatbots?
- Vibecoding
- Which AI to use?

What this lecture is NOT about

Too elementary (you already do this)

- AI as a search engine
- AI as a writing assistant or code explainer

Too advanced (not our focus today)

- Teams of AI agents coordinating with each other

Chatbots vs. agents

A **chatbot** responds in text. You copy-paste its output into your work.

An **agent** acts on your computer. It can:

- Read and write files
- Run Python and terminal commands
- Install packages
- Iterate until the task is done

Vibe coding

Term coined by Andrej Karpathy in February 2025:

> "There's a new kind of coding I call 'vibe coding', where you fully give in to the vibes, embrace exponentials, and forget that the code even exists."

Original tweet

Note: At the moment, the most successful vibe coders are people with deep technical knowledge. More on this here. However, it is hard to forecast since everything is changing rapidly

Vibe coding 2026

> “I ship code I don’t read.”

https://newsletter.pragmaticengineer.com/p/the-creator-of-clawd-i-ship-code

Which AI to use?

- The major models (Claude Sonnet 4.5, GPT-4o, Gemini) are more similar than different — you get 90% of the value from any of them
- Guide to choosing an AI (2025)

Getting the GitHub Copilot coding agent for free

The GitHub Copilot coding agent is a direct competitor to Claude Code — assign it a GitHub Issue, and it opens a pull request with the fix.

As a student, you can get it for free through **GitHub Education**:

1. Apply to GitHub Education using your university email
2. Wait for approval — **it can take a few days** for permissions to go through

Key differences from Claude Code: runs in GitHub's cloud (not your machine), operates on one issue at a time, and all

changes come back as a pull request for you to review.

Our stance

In all the other lectures, we were relatively opinionated about what you **should** do.

Here, our views are much less well-formed.

People are still working to understand what these tools can do, and what they **should** and **shouldn't** be allowed to do.

Also, first time ever giving this lecture so don't know pacing in advance, glad to go deeper with examples.

Summary

- A **chatbot** talks; an **agent** acts — it runs code, reads files, and iterates
- This shift is already happening: “vibe coding” is real and growing

Quick Start Guide

Roadmap

- Starting the agent
- Only runs in one directory
- Plan vs. execute mode
- `CLAUDE.md` — your persistent instructions
- Permissions
- Key shortcuts

Starting Claude Code

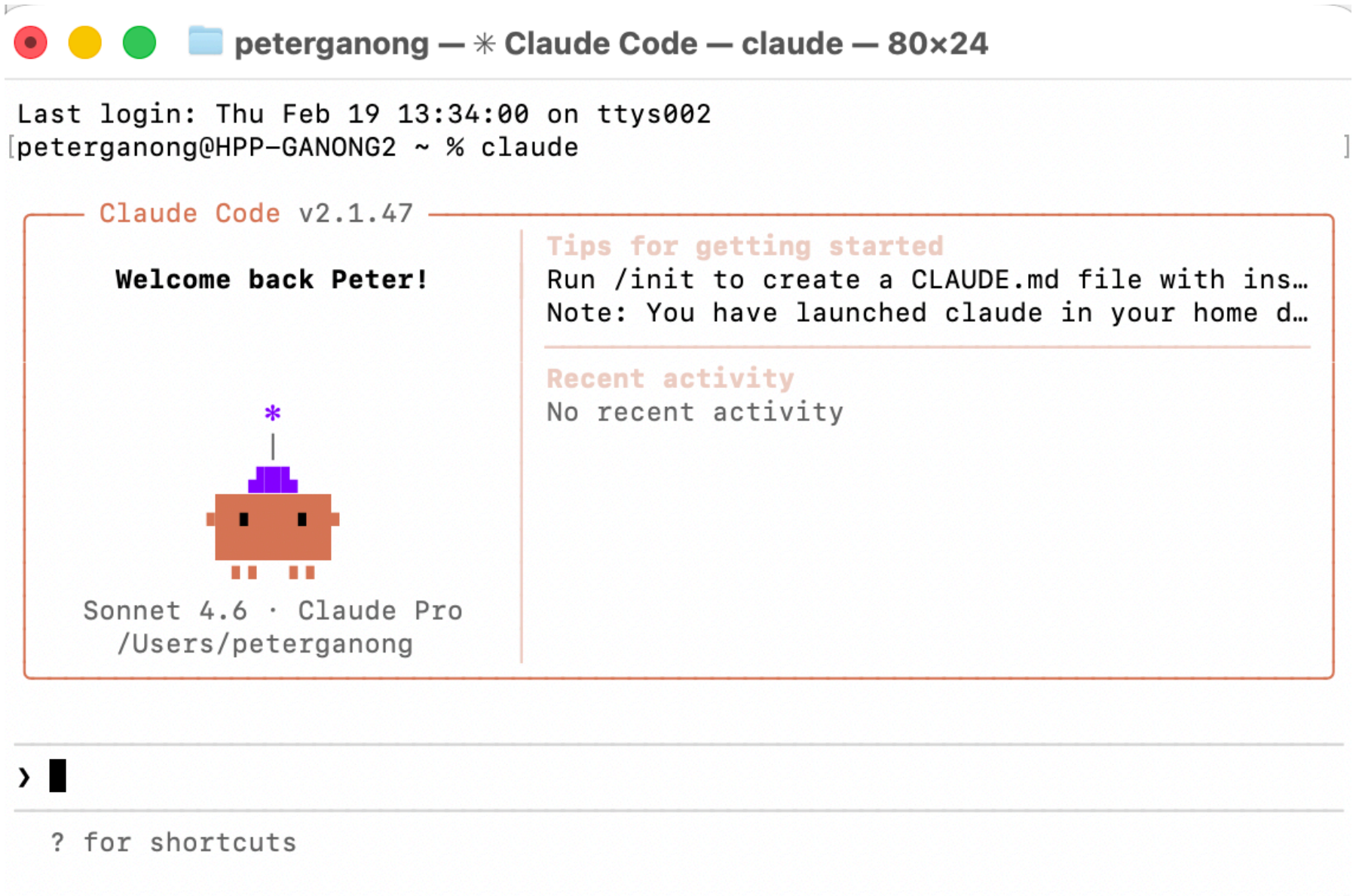
Download Claude Code. At the moment, a subscription is $20/month.

Open a terminal, navigate to your project folder, and type:

```
claude
```

That’s it. The agent is now running inside that folder.

What Claude Code looks like



One directory

The agent operates in **one directory** — the folder you started it in.

- It can read and write any file in that folder
- It cannot access files outside that folder by default
- This is a safety feature (more on this in Dangers)

One way to do this is to:

- Organize your project as a single folder before you start
- Use git in your agent folder

Permissions

By default, Claude asks for permission before running each terminal command.

```
Bash command

  git -C /Users/peterganong/repo/student30538-w26 log --oneline --all --
  "ps/ps2/data/parking_tickets_one_percent.csv" | head -5
  Find commits for largest file

Do you want to proceed?
❯ 1. Yes
  2. Yes, and don't ask again for similar commands in
     /Users/peterganong/repo/student30538-w26
  3. No
```

Leave this setting on. We explain why in the Dangers section.

Plan vs. execute mode

Agents have two modes:

Plan mode — Claude describes what it will do but does not act

- Use this to review the approach before letting it run
- Good for irreversible actions (deleting files, large rewrites)

Execute mode — Claude actually runs commands

- Toggle between modes with `Shift+Tab`

`CLAUDE.md` — your persistent instructions

Run `/init` in Claude Code to create a `CLAUDE.md` file.

This file persists across sessions. Claude reads it every time it starts.

Use it to tell Claude:

- What this project is
- Style preferences (e.g., “always use pandas, not polars”)
- What to avoid

CLAUDE.md — example

```
# Project
Analyze housing prices by metro area
using Census ACS data. Output goes
in output/figures/.

# Style
- Use pandas and altair (not polars/seaborn)
- Use relative paths everywhere
- Commit only code and data under 10MB

# Avoid
- Do not delete files in data/raw/
- Do not push to git without asking
```

Why this matters

- Claude reads this at the start of every session — no need to re-explain the project each time
- Enforces consistent style across all agent runs
- Sets guardrails on risky actions

Key shortcuts

Action	Shortcut
Exit Claude Code	`/exit`
Interrupt a running command	`Ctrl+C`
Toggle plan / execute mode	`Shift+Tab`
Get help	`/help`

Summary

- Type `claude` in your project folder to start
- The agent works in **one directory** — organize your project before you begin
- Use **plan mode** before irreversible actions; **execute mode** to run
- Write a `CLAUDE.md` to give the agent persistent instructions
- **Keep permissions on** — approve each command before it runs

Example 1: Analyze Insurance Status After Cancer Diagnosis

The task

Goal: Analyze how health insurance coverage changes after a cancer diagnosis — using ten years of a large public health survey.

My actual prompt

> Set up a GitHub repo. Download the last ten years of the MEPS. Identify people who get cancer or other serious diagnoses. Write Python code to track how often they subsequently lose their jobs and health insurance. Follow people over time in the MEPS. Once done, make a single Quarto document with all code and findings, knit to PDF.

Plan mode in action

Lazy user prompt

> analyze the meps data and make some charts about insurance loss after cancer, put it in a pdf

Claude's hypothetical plan (before acting)

1. Load `data/meps_pooled.csv`; inspect columns and year range
2. Filter to respondents with a new cancer diagnosis (`CANCERDX == 1`)
3. Merge across rounds to track the same person one year before and after diagnosis
4. Compute share uninsured at $t-1$, t, and $t+1$ by diagnosis severity
5. Produce three figures: event-study plot, bar chart by severity, coverage-type breakdown
6. Write `output/insurance_after_cancer.qmd` and knit to PDF

Waiting for approval before running any commands.

What the agent did (steps 1–4)

Step 1: Read the MEPS documentation to understand file structure

Step 2: Wrote a download script — fetched all ten years automatically

Step 3: Inspected the data, noticed that person identifiers changed format across years — fixed the merge key

Step 4: Ran the cleaning script — hit a missing package error, installed it, re-ran

What the agent did (steps 5–9)

Step 5: Identified newly diagnosed cancer patients using diagnosis codes

Step 6: Merged across survey rounds to track the same person over time

Step 7: Computed insurance loss rates by diagnosis severity

Step 8: Produced all main figures; noticed axis labels were inconsistent across plots — fixed them

Step 9: Wrote the Quarto document and knit it to PDF

Public repo

Why an agent beats a chatbot here

Chatbot

- Paste in the data (if it fits)
- Copy code out
- Run it yourself
- Paste errors back in
- Repeat 10–15 times

Agent

- Reads the data directly
- Runs the code
- Sees the error itself
- Fixes and re-runs
- Done

When to use an agent (vs. a chatbot)

Use an agent when:

- The task involves **multiple files** or steps
- You would otherwise copy-paste code and data back and forth
- Debugging requires context from the actual output

Stick to a chatbot when:

- You have a one-off conceptual question
- You want to understand code, not just run it

Summary

- One prompt can replace hours of copy-paste debugging
- Agents handle the full pipeline: download → clean → run → fix → visualize
- The key advantage: the agent sees errors in context and fixes them without your help
- Best for multi-step, iterative tasks; if you need to understand something yourself, regular self-study or a chatbot are more helpful

Example 2: Checking Replicability

The task

Question: Can a TA — or an AI agent — reproduce last year's final projects from scratch?

A project is **replicable** if someone else can:

1. Clone the repo
2. Install the required packages
3. Knit the `.qmd` file
4. Get outputs that match the submitted writeup

We gave the agent this task for six last-year final projects.

The prompt

> There are six student final projects linked from here

> Are they reproducible? You should be able to knit the .qmd file and re-generate one version of the writeup (HTML or PDF).

> For each project: 1. Check for a README. Download any data linked there. 2. Install required packages. 3. Knit the .qmd. Debug minor issues; stop after ~15 minutes. 4. Write a replication report to replication_reports/.md

What the agent did

For each of the six projects, the agent:

1. Read the README and located the `.qmd` file
2. Identified and installed required packages
3. Attempted to knit the `.qmd`
4. Debugged errors — fixing minor issues, stopping at major blockers
5. Wrote a structured report: data availability, execution status, output match, blockers

Then moved on to the next project.

Results

Project	Replicated?	Main blocker
Carjacking Analysis in Chicago	✅	—
Cooling Centers in DC	✅ (minor fix)	Malformed YAML syntax
Gang Violence in El Salvador	❌	Hardcoded paths + missing shapefiles
Quality of Life Index	❌	Incomplete shapefile + hardcoded path
Transportation and Public Safety	❌	Data not in repo, no download link
Vacancies and Neighborhood Economics	❌	QMD formatting bug + hardcoded path

2 of 6 replicated. All 6 are excellent projects — replication was simply not emphasized last year.

The #1 problem: hardcoded paths

Four of six projects had paths like these hardcoded into their `.qmd` files:

```
raw_data_path = r"C:\Users\User\OneDrive - The University of Chicago\..."
base_directory = '/Users/francescaleon/Documents/dap2_final_project/...'
directory      = r'C:\Users\laine\OneDrive\...'
```

These work on the author's machine and nowhere else.

Fix: use paths relative to the project folder:

```
raw_data_path = "data/traffic_crashes.csv"   # works anywhere
```

Other failure modes

Missing data files (2 projects)

- One project left data on a personal OneDrive with no public link
- One project committed only part of a shapefile (`.shp` only, no `.dbf`, `.prj`, `.cpg`)

Code syntax issues (2 projects)

What this means for your final project

This year, **full replicability is required**.

Checklist:

- Use **relative paths** everywhere — no hardcoded `/Users/...` or `C:\Users\...`
- Commit **all data files** to the repo, or provide a working download link in the README
- Test replication on a clean clone before submitting
- A TA (or an agent) should be able to run your `.qmd` without editing a single line

Summary

- Agents can run replication checks at scale — six projects, one prompt
- **2 of 6** last year's projects replicated cleanly
- The most common failure: hardcoded absolute paths
- Other failures: missing data files, subtle syntax errors
- This year: replicability is required — use relative paths and commit your data

Skills

What is a skill?

A **skill** is a pre-built capability you can give an agent — like a plugin or an app.

Examples:

- A skill that fetches live data from an API
- A skill that posts a chart to Slack
- A skill that auto-formats your code files

Skills extend what an agent can do beyond what's built in.

Skills documentation

How skills work

1. You install a skill (like installing a package)
2. The agent can call the skill during a task
3. The skill runs and returns a result to the agent

Skills are declared in `CLAUDE.md` so the agent knows they are available.

Using skills in Example 2

Installing skills

Skills are added by referencing them in your project configuration or via the agent's skill management.

Before installing any skill, ask:

- Who made this?
- What does it actually do?
- Does it need access to my files or network?

Important: this brings us to the next section.

Skills are external software

Installing a skill is like installing any software package.

You are trusting that it does what it says.

Most skills are fine. But some are not. The next section explains how to protect yourself.

Summary

- Skills are pre-built capabilities that extend what an agent can do
- They work like plugins: install once, the agent calls them as needed
- Useful for automating repetitive steps in a data analysis workflow
- **Treat them like any external software** — verify before installing

Dangers

Roadmap

- Risk 1: You accidentally delete sensitive files
- Risk 2: You let malicious software onto your computer
- Risk 3: You release sensitive data into the wild

Risk 1: Accidentally deleting files

> Someone let an AI agent manage files on their computer. It deleted 15 years of family photos — every picture of his wife.
>
> Tweet

This is not hypothetical. It happened.

Risk 1: The academic version

> “A professor from another institution, coauthoring with a UChicago professor, let an agent do a little bit too much. Specifically, they let an agent loose on the entire codebase. Did git save them? No, data was not saved with git!”

— Anecdote from an economics researcher at Booth

Risk 1: Solution

There's a setting — on by default — that requires your approval before every terminal command.

Leave it on.

The `--dangerously-skip-permissions` flag exists and some people use it. Here's why that's risky.

Do not use it unless you know exactly what you are doing.

Risk 2: Malicious skills

A “white hat” researcher demonstrated a real attack:

1. Released a skill for a popular AI agent desktop app
2. The skill secretly sent data from users’ computers to the researcher’s server
3. Falsely inflated download counts to appear as the most popular skill in the marketplace

Details

Risk 2: How to stay safe

- Only install skills from sources you trust — treat them like any software installation
- Read what a skill does before running it; check its source code if you can
- Keep default permissions on: Claude must ask before running commands
- The single-folder model limits blast radius if something goes wrong
- Be skeptical of very high download counts — popularity can be faked

Risk 3: Releasing sensitive data into the wild

In 2023, Samsung engineers pasted proprietary source code and internal meeting transcripts into ChatGPT to get help debugging.

The data was sent to OpenAI's servers — and potentially used to train future models.

Samsung banned ChatGPT company-wide shortly after.

Coverage

Risk 3: The research version

The same risk applies to your work:

- Student data with PII (names, IDs, survey responses)
- Confidential firm data shared under an NDA
- Preliminary research results not ready for public release
- Health records or administrative microdata with restricted use agreements

Use AI tools with caution when working with sensitive data. Make sure that it can’t leak your data.

Summary

- **Risk 1**: Agents can delete files — keep permissions on and back up your data with git
- **Risk 2**: Malicious skills can exfiltrate your data — only install from trusted sources
- **Risk 3**: Pasting sensitive data into AI tools sends it to external servers — know your data agreements
- This technology is evolving fast. Use it — carefully.

Going Deeper

MCPs: giving agents new tools

- **Model Context Protocol (MCP)** is a standard for connecting agents to external data sources and services (databases, APIs, browsers)
- Think of MCPs as a richer version of skills — they let the agent read live data or take actions outside its folder
- MCP documentation

Context windows and memory

- Every agent session has a **context window** — a limit on how much text the agent can “remember” at once
- When the window fills up, the agent automatically compresses earlier messages (“auto-compact”) — some detail is lost
- Manage this by starting fresh sessions for new tasks and keeping `CLAUDE.md` updated with key context
- At the end of a session, write a brief sign-out message summarizing what was done and what comes next — so the next session picks up without losing context

What are tokens?

- A **token** is the basic unit of text an LLM processes — roughly 4 characters or ¾ of a word
- Every message you send and receive consumes tokens; the model can only “see” what fits in its context window
- Larger files, longer histories, and verbose prompts all use more tokens
- **How to manage**: keep prompts concise, break large tasks into fresh sessions, avoid pasting entire large files when a snippet will do

Developing taste: what to delegate?

Key question: if I see the output, can I adequately judge whether it's right?

- **Easy to delegate** — success is obvious at a glance: web scraping, changing plot cosmetics, installing packages
- **Keep humans in the loop** — success requires deep understanding: data cleaning logic (example from quiz with CTA data – how to measure the income distribution of passengers), interpreting results (what conclusions actually follow from these patterns?)